

Data Specifications and Business Rules (v1.0)

Project: SplitFlow

Recipients: Backend Development Team

Role: Data Analyst / Product Analytics

Date: September 2026

1. Introduction and Objective

This document establishes the logical data architecture, server-side validation rules, and analytics event plan for the SplitFlow version. The objective is to ensure mathematical consistency in balance calculations, prevent financial rounding errors, and ensure clean product metric capture.

2. Schema and Data Dictionary (Data Model)

For relational database (SQL) persistence, the following main entities and their corresponding data types are defined:

2.1. USERS Table

- **user_id** (UUID, Primary Key, Not Null): Unique user identifier.
- **name** (VARCHAR 100, Not Null): User display name.
- **email** (VARCHAR 150, Unique, Not Null): Email address.
- **preferred_payout_alias** (VARCHAR 100, Nullable): Alias / CBU / CVU for receiving payments.
- **created_at** (TIMESTAMP, Not Null): Registration date (UTC).

2.2. GROUPS Table

- **group_id** (UUID, Primary Key, Not Null): Group identifier.
- **group_name** (VARCHAR 100, Not Null): Group/activity name.
- **created_by_user_id** (UUID, Foreign Key -> USERS.user_id, Not Null).
- **status** (VARCHAR 20, Not Null): Group status (active, settled).
- **created_at** (TIMESTAMP, Not Null).

2.3. GROUP_MEMBERS Table

- **group_id** (UUID, Foreign Key -> GROUPS.group_id, Not Null).
- **user_id** (UUID, Foreign Key -> USERS.user_id, Not Null).
- **joined_at** (TIMESTAMP, Not Null).

2.4. EXPENSES Table

- **expense_id** (UUID, Primary Key, Not Null).
- **group_id** (UUID, Foreign Key -> GROUPS.group_id, Not Null).
- **payer_user_id** (UUID, Foreign Key -> USERS.user_id, Not Null): User who made the payment.
- **title** (VARCHAR 150, Not Null): Expense concept/title.
- **total_amount** (DECIMAL 12,2, Not Null): Total expense amount (must be greater than 0).
- **split_type** (VARCHAR 20, Not Null): Split type (EQUAL, EXACT_AMOUNT).
- **created_at** (TIMESTAMP, Not Null).

2.5. EXPENSE_SPLITS Table

- **split_id** (UUID, Primary Key, Not Null).
- **expense_id** (UUID, Foreign Key -> EXPENSES.expense_id, Not Null).
- **user_id** (UUID, Foreign Key -> USERS.user_id, Not Null): Expense participant.
- **amount_owed** (DECIMAL 12,2, Not Null): Assigned individual share.

2.6. SETTLEMENTS Table

- **settlement_id** (UUID, Primary Key, Not Null).
- **group_id** (UUID, Foreign Key -> GROUPS.group_id, Not Null).
- **payer_user_id** (UUID, Foreign Key -> USERS.user_id, Not Null): Debtor user transferring funds.
- **receiver_user_id** (UUID, Foreign Key -> USERS.user_id, Not Null): Creditor user.
- **amount** (DECIMAL 12,2, Not Null): Amount paid.
- **status** (VARCHAR 20, Not Null): Payment status (CONFIRMED, PENDING).
- **settled_at** (TIMESTAMP, Not Null).

3. Business Rules and Server-Side Validations

The Backend must implement and validate the following logical rules before executing any database transaction:

1. Zero-Sum Validation in Expenses:

For any expense creation or editing, the sum of the assigned shares must be strictly equal to the total recorded amount:

$$\sum \text{EXPENSE_SPLITS.amount_owed} = \text{EXPENSES.total_amount}$$

If there is a discrepancy, the API must reject the payload with an HTTP 400 (BAD_REQUEST) error code.

2. Decimal and Odd Number Handling:

- The fixed-point data type `DECIMAL (12, 2)` must be used for all monetary variables.
- In equal splits with a decimal remainder (e.g., $\$10,000 / 3 = 3,333.333\dots$), the leftover cent or remainder (0.01) will be automatically assigned to the user who paid the expense (`payer_user_id`).

3. Dynamic Net Balance Calculation:

The individual balance within a group is calculated using the formula:

$$\text{Net Balance} = \text{Amount Paid} - \text{Amount Consumed} + \text{Payments Received} - \text{Payments Made}$$

- If Net Balance > 0, the user is a Creditor (they are owed money).
- If Net Balance < 0, the user is a Debtor (they owe money).

4. Debt Simplification Algorithm:

- The system will consolidate net balances among members to reduce the number of necessary transfers (debt graph optimization).

4. Analytics Event Plan (Tracking Plan)

To measure user experience performance (UX Analytics), the Backend / Frontend must send the following structured events to the analytics platform:

Event	Trigger	Mandatory Properties
group_created	Successful creation of a group	group_id (UUID), created_by_user_id (UUID)
expense_created	Registration of a new expense	expense_id (UUID), group_id (UUID), total_amount (DECIMAL), split_type (STRING), participants_count (INT)
settlement_confirmed	Confirmation of payment between users	settlement_id (UUID), group_id (UUID), payer_id (UUID), receiver_id (UUID), amount (DECIMAL)
group_settled	When all group balances reach 0.00	group_id (UUID), total_expenses_count (INT), total_volume_amount (DECIMAL)

5. Quality Control (QA) Next Steps

1. **Synthetic Dataset Generation:** Creation of test data with edge cases (groups of 10+ members, odd amounts, circular debts).
2. **Validation Matrix:** Execution of stress tests on calculation endpoints to verify consistency prior to production deployment.